

Backend Engineering Code Challenge

Payment Processing with Fraud Detection

Overview

In this challenge, you'll build a Payment Processing system for a digital bank that processes transactions, applies fraud detection rules, and maintains audit logs. The system must handle concurrent transactions, manage state correctly, and provide observability.

This is a real-world backend engineering problem that tests your ability to:

- Design clean, layered architecture
- Make trade-off decisions (sync vs. async, where logic lives, etc.)
- Handle data consistency and concurrency
- Build testable code with good coverage
- Write clear documentation and commit messages

Business Requirements

Requirement	Details
Process Transactions	Accept transaction requests with: amount, user ID, merchant ID, and category. Assign a unique transaction ID and timestamp. Return either a successful confirmation (APPROVED/FLAGGED) or an error (DECLINED).
Fraud Detection Engine	Rule 1: DECLINE if amount > 10,000 without prior user approval Rule 2: DECLINE if >3 transactions from same user within 60 seconds Rule 3: DECLINE if high-risk category AND amount > 5,000 Rule 4: FLAG (but allow) if amount > 5,000 from user created < 30 days ago
Audit Logging	Persistently log all transactions with: decision (APPROVED/DECLINED/FLAGGED), rules triggered, timestamp, and user context. Audit logs must survive service restarts.
User Management	Track user creation date, KYC status, and approval limits. Provide REST endpoints to query and update user profiles.
Resilience	If a downstream service (user database, audit log storage) is slow or unavailable, gracefully degrade. Never lose transaction data.

Technical Requirements

Stack & Tools:

- Java Spring Boot (3.x or 2.7 LTS)
- Maven for build management
- PostgreSQL for transactional data (transactions, users)
- MongoDB for audit logs (or PostgreSQL if you prefer single DB)
- Docker & Docker Compose (all services start with single command)

Code Quality:

- Structured logging (SLF4J + Logback, NOT System.out.println)
- Clean REST API with proper HTTP status codes and error responses
- Clear separation of concerns (controller, service, repository layers)
- Object-oriented design with meaningful class hierarchies

Testing:

- Unit tests using JUnit 5 and Mockito (minimum 75% code coverage)

- Integration tests with TestContainers (or embedded DB) for database layers
- All tests pass and coverage reports are generated

Documentation:

- README.md with setup, build, run, and test instructions
- API documentation (Swagger/OpenAPI or detailed examples in README)
- UML class diagram of domain model (User, Transaction, FraudRule, AuditLog)
- Clear, meaningful Git commit messages

Deliverables Checklist

- ☐ GitHub repository (public or private, share the link)
- ☐ All source code committed with clean commit history
- ☐ docker-compose.yml that runs everything (Spring app, PostgreSQL, MongoDB)
- ☐ pom.xml with all dependencies and build plugins
- ☐ All 4 fraud detection rules implemented and working
- ☐ Audit logging functional (transactions logged to MongoDB or PostgreSQL)
- ☐ User management endpoints (create, query, update users)
- ☐ Transaction processing endpoint (POST /transactions or similar)
- ☐ Unit tests (>75% coverage, all passing)
- ☐ Integration tests (at minimum for DB layers)
- ☐ Code coverage report (JaCoCo or similar)
- ☐ README.md with full setup & run instructions
- ☐ API documentation (Swagger YAML or examples)
- ☐ UML class diagram (included in README or separate file)

Optional Enhancements:

- Idempotency keys (prevent duplicate transactions)
- Webhook/notification system for transaction events
- Rate limiting per user or merchant
- Bulk transaction export endpoint with pagination
- Redis caching for user data or fraud rules
- SonarQube or static code analysis integration

Key Design Decisions You'll Make

You will need to make thoughtful choices about:

Architecture: Where should fraud rules live? In a service? In a separate rules engine? How do you make them testable and reusable?

Audit Logging: Should audit logs be written synchronously (blocking) or asynchronously (fire-and-forget)? What are the trade-offs?

Concurrency: How do you safely handle multiple transactions from the same user arriving at the same time? What race conditions exist?

Data Consistency: If PostgreSQL and MongoDB are both involved, how do you ensure they stay consistent? What happens if one is down?

Resilience: If the user database is slow, should that block transaction processing? How do you degrade gracefully?

Assessment Criteria (What We're Looking For)

This challenge evaluates:

- ✓ Correctness: All fraud rules work correctly, audit logs are captured, no crashes on edge cases
- ✓ Architecture: Clean layers, separation of concerns, testable design
- ✓ Testing: Good coverage (75%+), both unit and integration tests, smart use of mocks
- ✓ Data Handling: Concurrent transaction handling, consistency across databases, audit trail integrity
- ✓ Communication: Clear code, good documentation, thoughtful commit messages

A Note on AI-Assisted Development

You can use AI tools. We expect that in 2024. However:

- Your solution must be yours. You need to understand every architectural decision.
- In the technical interview, you'll explain your code, make live changes, and answer follow-up questions. If you don't understand your own solution, it will show.
- Focus on trade-offs and design decisions—those are harder to generate and easier to explain.

README.md Template

Your README should include:

Payment Processing System

```
## Overview [1-2 sentences describing the system]
## Architecture [Brief description of layers, key components, design decisions]
## Setup & Prerequisites [Java version, Maven, Docker, etc.]
## Running the Application [Step-by-step: clone, build, docker-compose up]
## API Endpoints [List with examples: POST /transactions, GET /users/:id, etc.]
## Running Tests [mvn test, coverage report generation]
## Code Coverage [How to view JaCoCo report]
## Design Decisions [Why you chose certain approaches; trade-offs considered]
```

GitHub & Submission

- Create a public or private GitHub repository
- Repository name should be generic (e.g., "payment-processing-system" or "fraud-detection-service"—avoid any company names)
- Commit your work regularly with clear, descriptive messages

Next Steps

1. Read through this entire document carefully. Note any ambiguities and ask clarifying questions during the discovery session.
2. Sketch out your architecture on paper or a whiteboard before coding.
3. Start with Docker setup and database schema—these are foundational.
4. Implement core features first (transaction processing, fraud rules, logging).
5. Write tests as you go—easier than retrofitting.
6. Save time for documentation and README.
7. Push to GitHub and submit the link on time.

Good luck! We look forward to reviewing your work.